UNIVERSITY OF SCIENCE AND TECHNOLOGY OF HANOI

DEPARTMENT OF INFORMATION COMMUNICATION AND TECHNOLOGY

--------***--------

REPORT LABWORK

MACHINE LEARNING AND DATA MINING II

CLASS: 1

	STUDENT NAME	STUDENT ID
1	Nguyễn Quang Hưng	22BI13184
2	Lưu Anh Đức	22BI13088

Ha Noi, February 9th 2025

Data: For this labwork, we choose 2 dataset from Kaggle.

Repository namely Titanic dataset and Iris dataset.

I. TITANIC DATASET

1. Statistical exploration

First, we read the titanic data into a dataframe using pandas (a package for exploring data in Python)

1.1 Dataset Overview

- **Total Observations**: 891 passengers
- **Features**: 12 original features
- **Target Variable**: Survival status(0: Did not survive, 1: Survived)

1.2 Feature Classification

Feature	Discrete/ Continuous	Quantitative/ Qualitative	Numerical/ Categorical	Description
PassengerId	Discrete	Quantitative	Numerical	Id for each passenger
Survived	Discrete	Quantitative	Categorical	Survival outcome (0 = No, 1 = Yes)
Pclass	Discrete	Quantitative	Categorical	Passenger Class (1st, 2nd, 3rd)
Name	Discrete	Quantitative	Categorical	Passenger's full name
Sex	Discrete	Quantitative	Categorical	Gender of passenger
Age	Continuous	Quantitative	Numerical	Passenger's age
SibSp	Discrete	Quantitative	Numerical	Number of siblings/spouses aboard
Parch	Discrete	Quantitative	Numerical	Number of parents/children aboard
Ticket	Discrete	Quantitative	Categorical	Ticket number
Fare	Continuous	Quantitative	Numerical	Passenger ticket fare
Cabin	Discrete	Quantitative	Categorical	Cabin number

Embarked	Discrete	Quantitative	Categorical	Port of embarkation

1.3 Checking the issues lying within the data

The data was checked missing data with the following command:

```
print(data.isnull().sum())
```

After checking, there are some features missing values:

```
PassengerId      0
Survived         0
Pclass           0
Name             0
Sex              0
Age            177
SibSp            0
Parch            0
Ticket           0
Fare             0
Cabin          687
Embarked         2
dtype: int64
```

Age: 177 missing values (19.9% of dataset)

Cabin: 687 missing values (77.1% of dataset)

Embarked: 2 missing values (0.2% of dataset)

1.4 Preprocessing Steps

There are some missing values that need to be prepared for exploratory data analysis :

- Age: Median imputation

```
data['Age'].fillna(data['Age'].median(), inplace=True)
```

- Embarked: Mode imputation

```
data['Embarked'].fillna(data['Embarked'].mode()[0], inplace=True)
```

- Cabin: Dropped due to high missingness

```
data.drop('Cabin', axis=1, inplace=True)
```

1.5 Exploratory Data Analysis

There are 4 values that need to be calculated to perform analysis on this dataset: mean, variance, covariance, correlation. Before that, we need to choose numerical features to calculate. For this dataset, ‘Age’,’Fare’, ‘SibSp’, ‘Parch’ are numerical feature.

Mean:

Mean values tell us the average of each numerical feature:

```
numerical_features = data[['Age', 'Fare', 'SibSp', 'Parch']]
mean_values = numerical_features.mean()
print("Mean:\n", mean_values)
```

Output:

```
Mean:
 Age       29.361582
Fare      32.204208
SibSp      0.523008
Parch      0.381594
dtype: float64
```

Variance:

Variance shows the spread of the data

```
variance_values = numerical_features.var()
print("Variance:\n", variance_values)
```

Output:

```
Variance:
 Age        169.512498
Fare      2469.436846
SibSp        1.216043
Parch        0.649728
dtype: float64
```

Covariance:

Covariance indicates how features vary together

```
covariance_matrix = numerical_features.cov()
print("Covariance:\n", covariance_matrix)
```

Output:

```
Covariance:
               Age         Fare     SibSp     Parch
Age    169.512498    62.556767 -3.349525 -1.810133
Fare    62.556767  2469.436846  8.748734  8.661052
SibSp   -3.349525     8.748734  1.216043  0.368739
Parch   -1.810133     8.661052  0.368739  0.649728
```

Correlation:

Correlation shows the strength and direction of relationships

```
correlation_matrix = numerical_features.corr()
print("Correlation:\n", correlation_matrix)
```

Output:

```
Correlation:
             Age      Fare     SibSp     Parch
Age     1.000000  0.096688 -0.233296 -0.172482
Fare    0.096688  1.000000  0.159651  0.216225
SibSp  -0.233296  0.159651  1.000000  0.414838
Parch  -0.172482  0.216225  0.414838  1.000000
```

After calculated 4 values, we need to find the most corelated couple of features of each dataset by using correlation matrix.

To find the most strongly correlated feature pair, we will:

- Remove the values on the main diagonal (since they are always 1).
- Find the largest value (close to 1 or -1) in the correlation matrix.

```
correlation_matrix = numerical_features.corr()
print("Correlation:\n", correlation_matrix)
correlation_matrix.values[np.arange(len(correlation_matrix)), np.arange(len(correlation_matrix))] = 0
max_corr = correlation_matrix.max().max()
max_corr_pair = np.where(correlation_matrix == max_corr)
features = correlation_matrix.columns
max_feature1, max_feature2 = features[max_corr_pair[0][0]], features[max_corr_pair[1][0]]
print(f"Most positively correlated pair: ({max_feature1}, {max_feature2}) with correlation {max_corr:.4f}")
```

Output:

```
Most positively correlated pair: (SibSp, Parch) with correlation 0.4148
```

Conclusion for the most corelated couple:

- Feature pair: SibSp (number of accompanying siblings/spouse) and Parch (number of accompanying parents/children).
- Correlation value: 0.4148.
- Comments: The positive correlation shows that as the number of accompanying siblings/spouse increases, the number of accompanying parents/children also tends to increase. This makes sense since families often travel together on trains.

2. Principal Component Analysis (PCA)

2.1 Data preparation

Before applying PCA, we need to:

- Handle missing values.
- Standardize the data (because PCA is sensitive to the scale of the features).

2.2 Apply PCA

PCA will transform the data from the original space to a new space with the axes (principal components) ordered by importance (variance).

```
pca = PCA()
pca_data = pca.fit_transform(scaled_data)
explained_variance_ratio = pca.explained_variance_ratio_
```

Output:

```
Explained Variance Ratio:
 [0.40662092 0.27577972 0.17298644 0.14461292]
```

2.3 Select the number of principal components

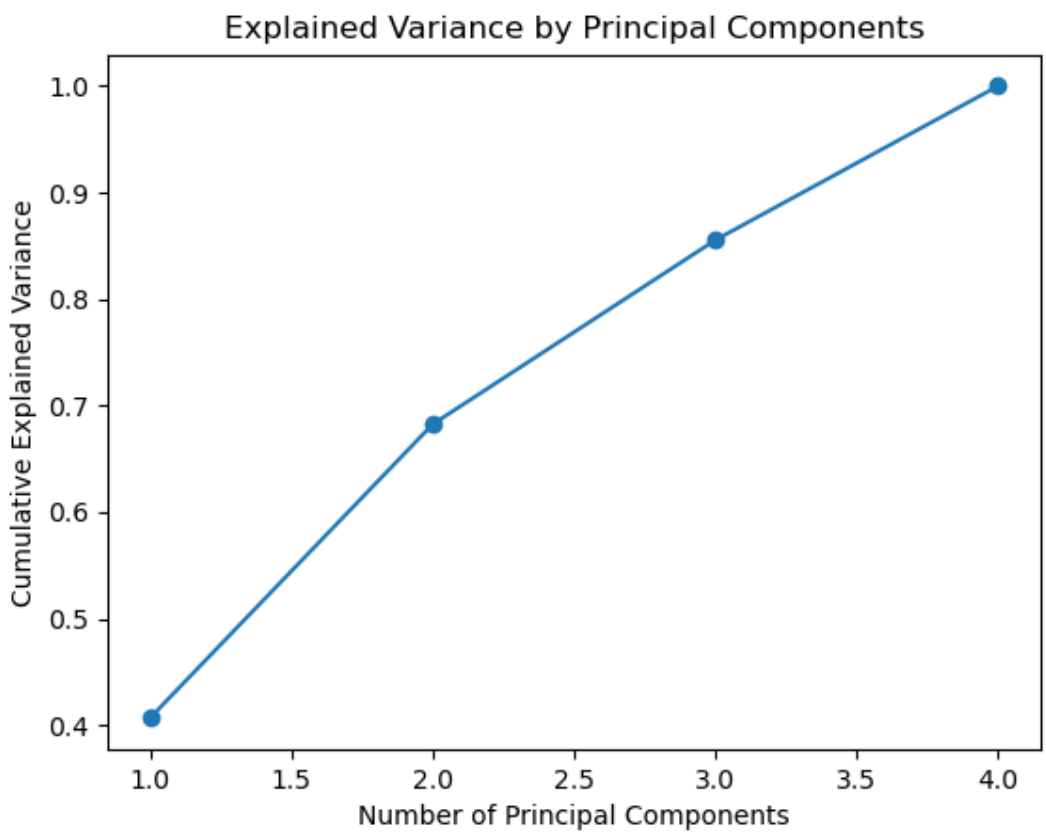
The principal components are arranged in decreasing order of variance. We can select the number of principal components based on the cumulative explained variance. For example, We use 4 principal components:

```
pca = PCA()
pca_data = pca.fit_transform(scaled_data)
explained_variance_ratio = pca.explained_variance_ratio_
print("Explained Variance Ratio:\n", pca.explained_variance_ratio_)

cumulative_variance = np.cumsum(explained_variance_ratio)

plt.plot(range(1, len(cumulative_variance) + 1), cumulative_variance, marker='o')
plt.xlabel('Number of Principal Components')
plt.ylabel('Cumulative Explained Variance')
plt.title('Explained Variance by Principal Components')
plt.show()
```

Output:



The graph will show the cumulative variance increasing as the number of principal components increases.

2.4 Principal Components Variance Analysis

Principal component with the highest value (PC1):

- Contains the most information about the data (largest variance).
- Usually represents the main trend of the data.

Principal component with the lowest value (PC4):

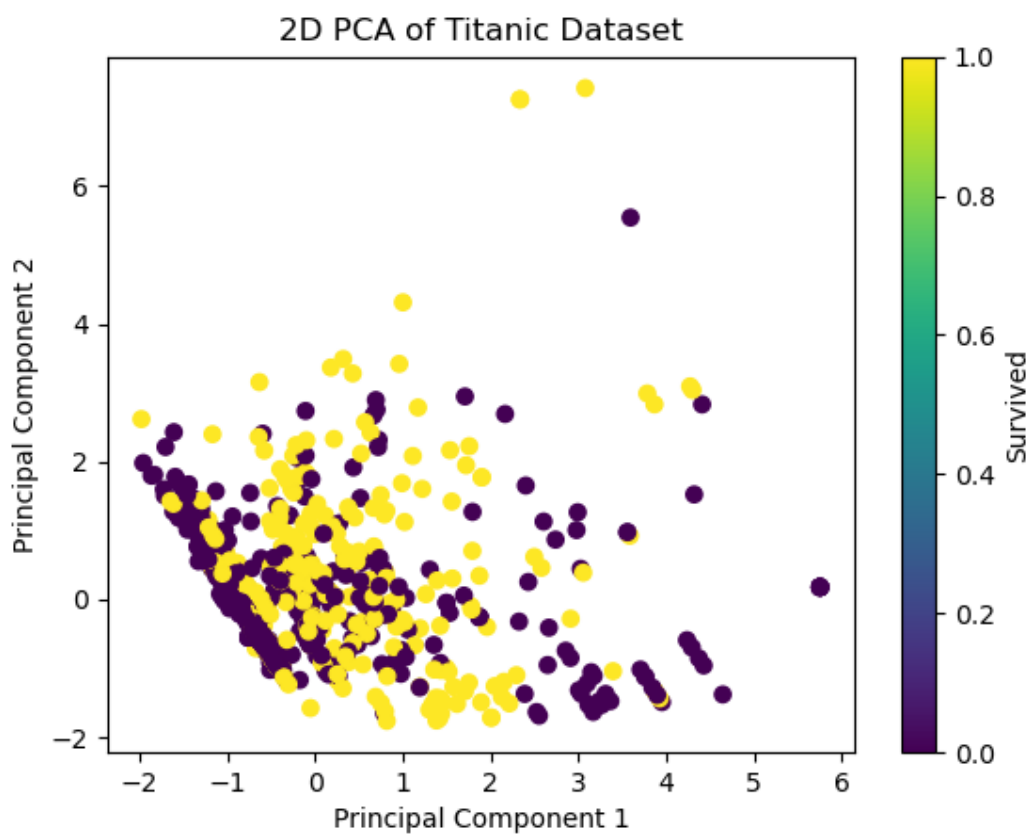
- Contains the least information (smallest variance).
- Usually represents noise or unimportant information.

2.5 Visualize data in 2D space

Use the first two principal components to plot the data distribution:

```
plt.scatter(pca_data_2d[:, 0], pca_data_2d[:, 1], c=data['Survived'], cmap='viridis')
plt.xlabel('Principal Component 1')
plt.ylabel('Principal Component 2')
plt.title('2D PCA of Titanic Dataset')
plt.colorbar(label='Survived')
plt.show()
```

The chart will display the data points in 2D space, with colors representing the Survived label:



2.6 Conclusion

PCA helps reduce the dimensionality of data while preserving the main structure. The fewer the number of principal components, the faster the processing speed, but the accuracy may decrease.

Visualizing data in 2D space helps to observe the distribution and relationship between data points.

II. IRIS DATASET

1. Statistical exploration

1.1 Dataset

We use *pandas* to read the Iris data into a dataframe.

The dataset is directly downloaded from the URL and does not need a local file.

```
# Import libraries
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
```

```
# Load the dataset
url = "https://raw.githubusercontent.com/uiuc-cse/data-fa14/gh-pages/data/iris.csv"
data = pd.read_csv(url)
```

1.2 Features

Feature	**Discrete/ Continuous**	**Quantitative/ Qualitative**	**Numerical/ Categorical**
Sepal length	Continuous	Quantitative	Numerical
Sepal width	Continuous	Quantitative	Numerical
Petal length	Continuous	Quantitative	Numerical
Petal width	Continuous	Quantitative	Numerical
Species	Discrete	Qualitative	Categorical

This dataset records 5 features for all Iris samples that are divided into 3 groups (Setosa, Versicolour, Virginica). Except "Species" to identify Iris samples, 4 features left are:

- **Continuous**: they can be any real value.
- **Quantitative:** they can be ranked and their value is **numerical**.

1.3 Check the issue of data

Checking missing data:

```
# Check for missing values
print("\nMissing Values:\n", data.isnull().sum())
```

After running above command, the missing data will show:

```
Missing Values:
 sepal_length    0
sepal_width     0
petal_length    0
petal_width     0
species         0
dtype: int64
```

We can see that there is no data missing here. Then, we can move to the next process.

1.4 Exploratory Data Analysis

We use *numpy* to identify variance, covariance, correlation, mean.

Variance

The variance (σ^2) is the dispersion of a random variable

```
np.var(data.sepal_length), np.var(data.petal_width)
```

Output:

```
(0.6811222222222222, 0.5785315555555559)
```

Covariance

The covariance is the tendency of two variables (features) to vary together

```
# The covariance between Sepal Length and Petal Length
np.cov(data.sepal_length, data.petal_length)

# The covariance between Sepal Length and Petal Width
np.cov(data.sepal_length, data.petal_width)
```

Output:

```
array([[0.68569351, 1.27368233],
       [1.27368233, 3.11317942]])

array([[0.68569351, 0.5169038 ],
       [0.5169038 , 0.58241432]])
```

Correlation

Correlation plays a foundational role in PCA which measures the strength and direction of a linear relationship between two variables. Correlation can be present in the covariance matrix.

For m-features, we can express it in a $m \times m$ covariance matrix

```
def covariance_matrix(X):
    n = X.shape[0]
    Z = np.divide(X - X.mean(axis=0), X.std(axis=0))

    return np.matmul(Z.T, Z)/(n-1)

    S = covariance_matrix(X)
    S
```

Output:

```
array([[ 1.00671141, -0.11010327,  0.87760486,  0.82344326],
       [-0.11010327,  1.00671141, -0.42333835, -0.358937  ],
       [ 0.87760486, -0.42333835,  1.00671141,  0.96921855],
       [ 0.82344326, -0.358937  ,  0.96921855,  1.00671141]])
```

Mean

```
# Center the data around the mean of the observation
Z_score = np.divide(X- X.mean(axis=0), X.std(axis=0)) # substract mean and divide by variance
Z_score.shape
S = np.cov(Z_score, rowvar=False, bias=False)
S
```

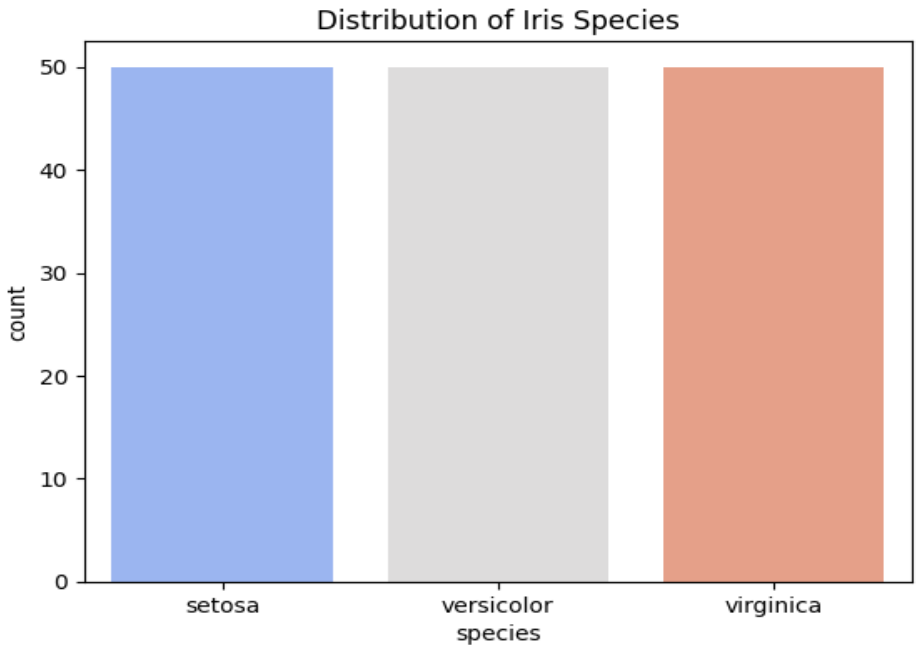
Output:

```
array([[ 1.00671141, -0.11010327,  0.87760486,  0.82344326],
       [-0.11010327,  1.00671141, -0.42333835, -0.358937  ],
       [ 0.87760486, -0.42333835,  1.00671141,  0.96921855],
       [ 0.82344326, -0.358937  ,  0.96921855,  1.00671141]])
```

*** Calculate these measures for species feature**

```
# Target distribution
print(data.columns)
sns.countplot(x='species', data=data, palette='coolwarm')
plt.title('Distribution of Iris Species')
plt.show()
```

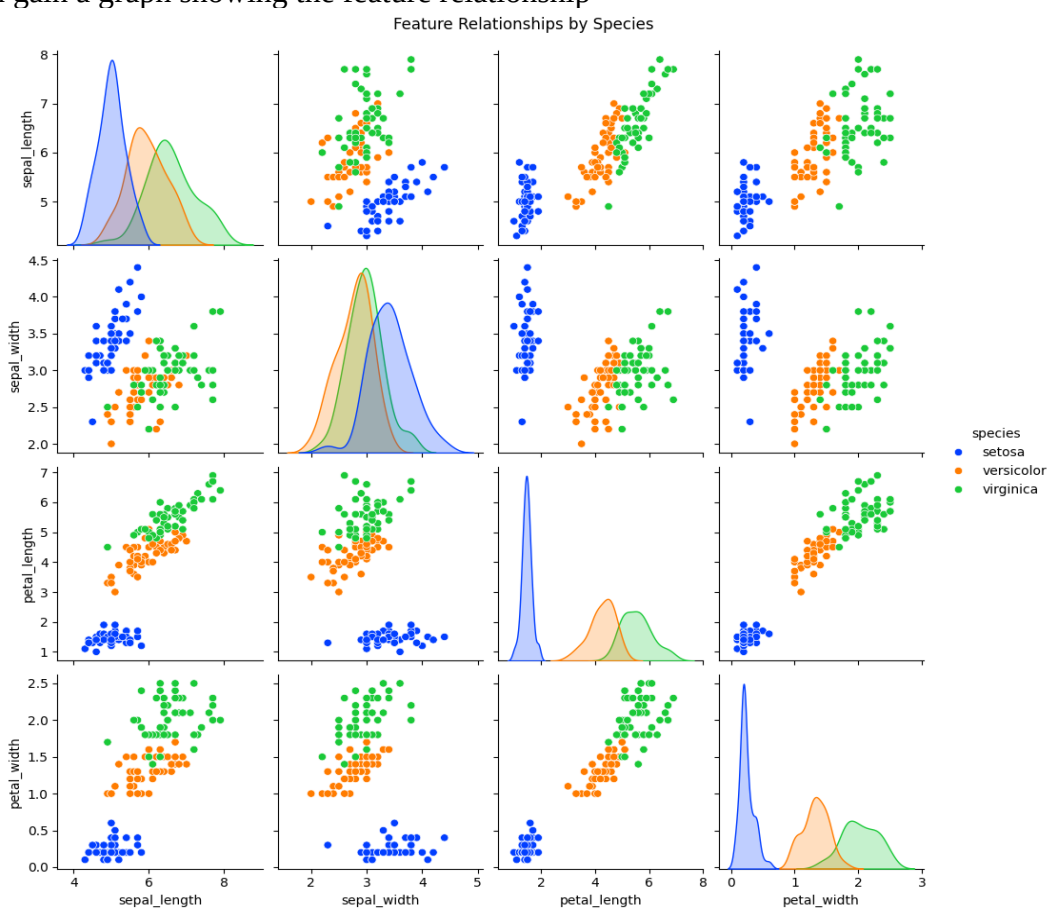
Output:



Then, we visualize relationships between features using *seaborn* and *matplotlib*

```
sns.pairplot(data, hue='species', diag_kind='kde', palette='bright')
plt.suptitle('Feature Relationships by Species', y=1.02)
plt.show()
```

Finally, we can gain a graph showing the feature relationship



2. Principal Component Analysis (PCA)

2.1 Apply PCA

We initialize PCA using *sklearn.decomposition* library

```
from sklearn.decomposition import PCA
myPCA = PCA(n_components = 2, svd_solver = 'full')
T = myPCA.fit_transform(Z)

var = myPCA.explained_variance_ratio_*100
plt.scatter(T[:,0], T[:,1], c = species, s = 14)
plt.xlabel(f'PC$_1$ = {var[0]:2.1f} %', fontsize = 14);
plt.ylabel(f'PC$_2$ = {var[1]:2.1f} %', fontsize = 14);
```

- n_components = 2: reduce the data to 2 principal components, PCA will use *SVD decomposition* to calculate all components.
- Select the principal components:

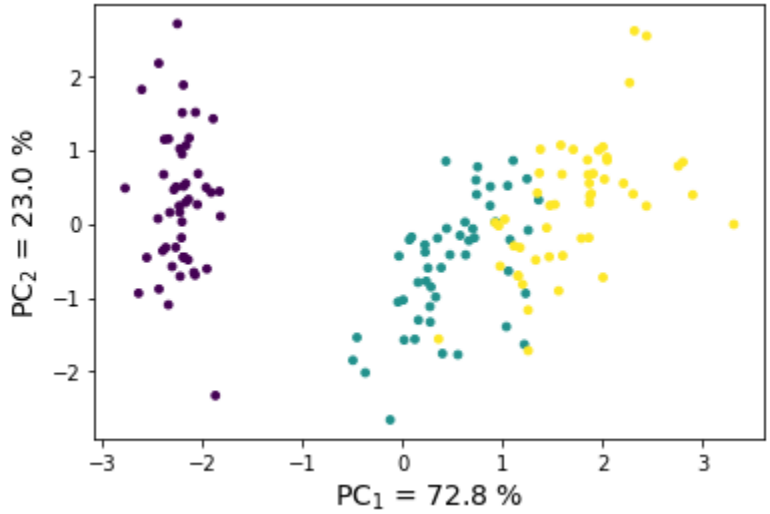
  + *Computing the covariance matrix:* PCA standardizes the data and then computing the covariance matrix of the data.

  + *Identify eigenvalues and eigenvectors:* Based on covariance matrix, PCA extracts eigenvalues (corresponds to a principal component) and eigenvalue (indicates the amount of variance).
- Transform the data: Find the principal components from data Z and transform it into new space composed of 2 principal components. Matrix T contains PC1 and PC2, which are the two components with the highest explained variance.
- Compare highest and lowest values:

  + *Highest value components:* Retain most of the important information for dimensionality reduction and data analysis.

  + *Lowest value components:* Capture minor details which are generally not used for analysis since they do not contribute to understanding the data structure.

2.2 Obtained result



- Using Two Principal Components PC1 and PC2, the current visualization retains 95.8% (72.8% + 23.0%) of the variance. It balances both problems of dimensionality reduction and information retention.

- Two Principal Components is the optimal method, adding a third principal component will increase the total variance explained, however, the change is not much and visualization beyond 3D is more complicated.